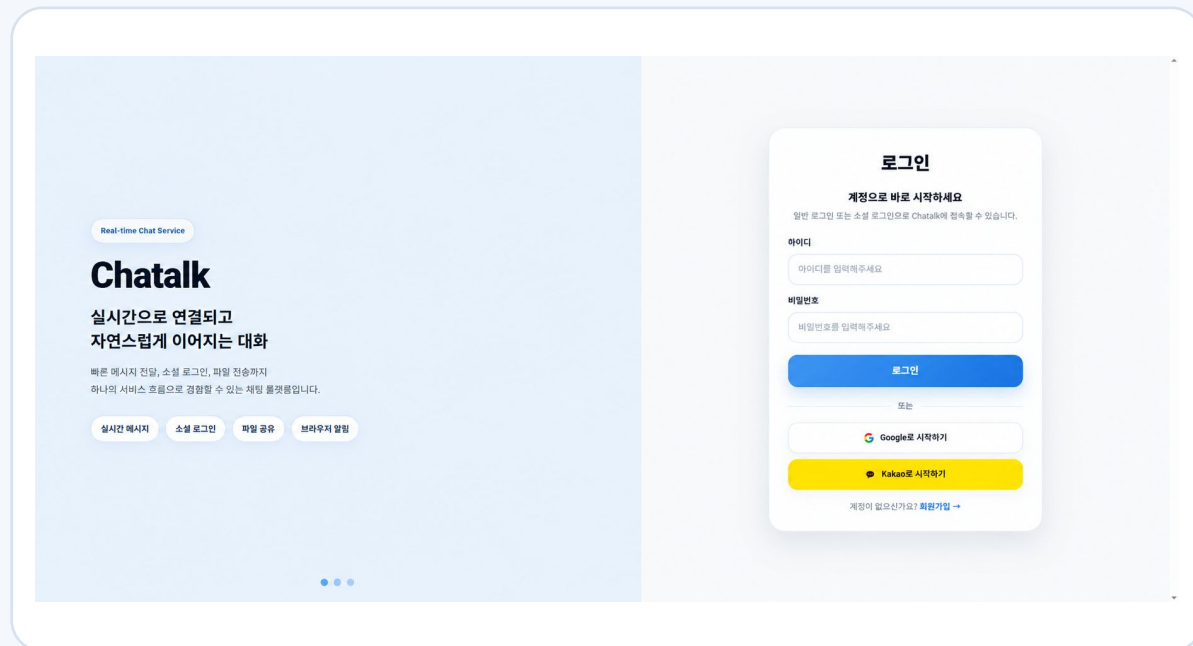


Chataalk

실시간 채팅 서비스 요약 포트폴리오

WebSocket/STOMP 기반 실시간 메시징부터 읽음 정합성, 재연결 복구, DB 메타데이터 경합 개선, 파일·검색 처리까지 실제 서비스 흐름을 기준으로 설계하고 검증한 개인 프로젝트입니다.

Java 21 Spring Boot WebSocket/STOMP Redis MySQL Vue Docker



개발 기간: 2025.12 ~ 진행 중
개발 형태: 개인 프로젝트
배포 URL: <https://chataalk.store>

[박민수 | Backend Developer](#)
[GitHub: https://github.com/minsu11/live_chat](https://github.com/minsu11/live_chat)
[상세 자료: Notion / 상세 포트폴리오](#)

프로젝트 한눈에 보기

실시간 채팅 서비스에서 발생하는 정합성, 복구, 경합 문제를 중심으로 구현했습니다.

46 / 60 → 60 / 60

동일 채팅방 DB 저장

300명

high 시나리오 연결

3개 저장소

API / Auth / Front

진행 중

개발 기간: 2025.12 ~ 진행 중

핵심 구현 범위

- WebSocket/STOMP 기반 실시간 메시지 송수신
- lastReadMessageId 기반 읽음/안읽음 정합성
- afterMessageId 기반 재연결 후 누락 메시지 복구
- Redis Pub/Sub + Write-Back + Dirty Set + Bulk Update
- 파일 선업로드와 orphan attachment cleanup
- Full-Text Search + LIKE fallback 검색 UX

담당 역할

- API 서버: 채팅방, 메시지, 읽음, 파일, 검색 도메인 설계/구현
- Auth 서버: 일반 로그인, OAuth2, JWT/쿠키 인증 처리
- Frontend: Vue 기반 채팅 UI, 검색 하이라이트, 스크롤 보정
- Infra: Docker 배포, Nginx Reverse Proxy, Cloudflare 연결
- 검증: publish / DB 저장 / WebSocket 수신 비교

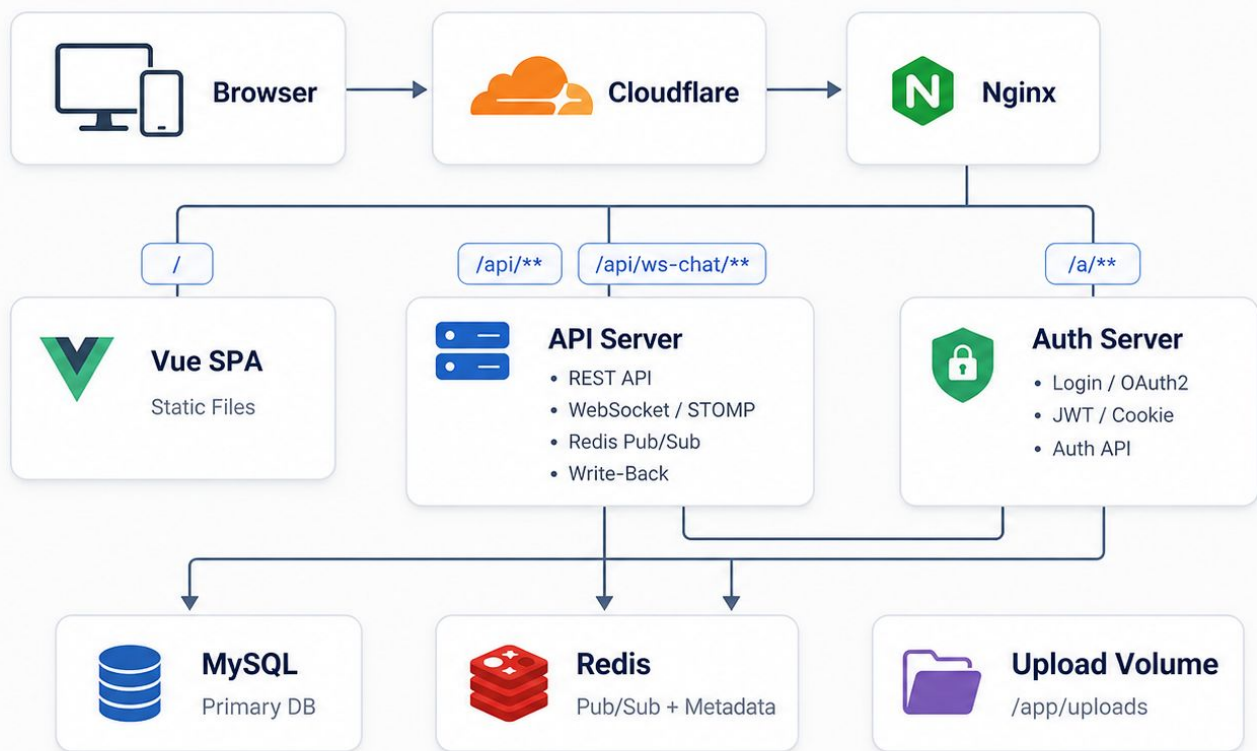
핵심 요약: 단순 채팅 UI가 아니라, 메시지 유실 / 읽음 정합성 / DB update 경합 / 파일 orphan / 검색 UX 문제를 실제 서비스 흐름 기준으로 해결한 프로젝트입니다.

전체 배포 아키텍처

Cloudflare와 Nginx를 통해 요청을 라우팅하고, API/Auth 서버와 MySQL·Redis·Upload Volume으로 서비스를 구성했습니다.

전체 배포 아키텍처

Nginx 경로 기반 라우팅과 Redis Pub/Sub 기반 실시간 전파 구조



요청 라우팅

- / : Vue SPA 정적 파일 서빙
- /api/** : REST API 프록시
- /api/ws-chat/** : WebSocket /STOMP 프록시
- /a/** : Auth 서버 프록시

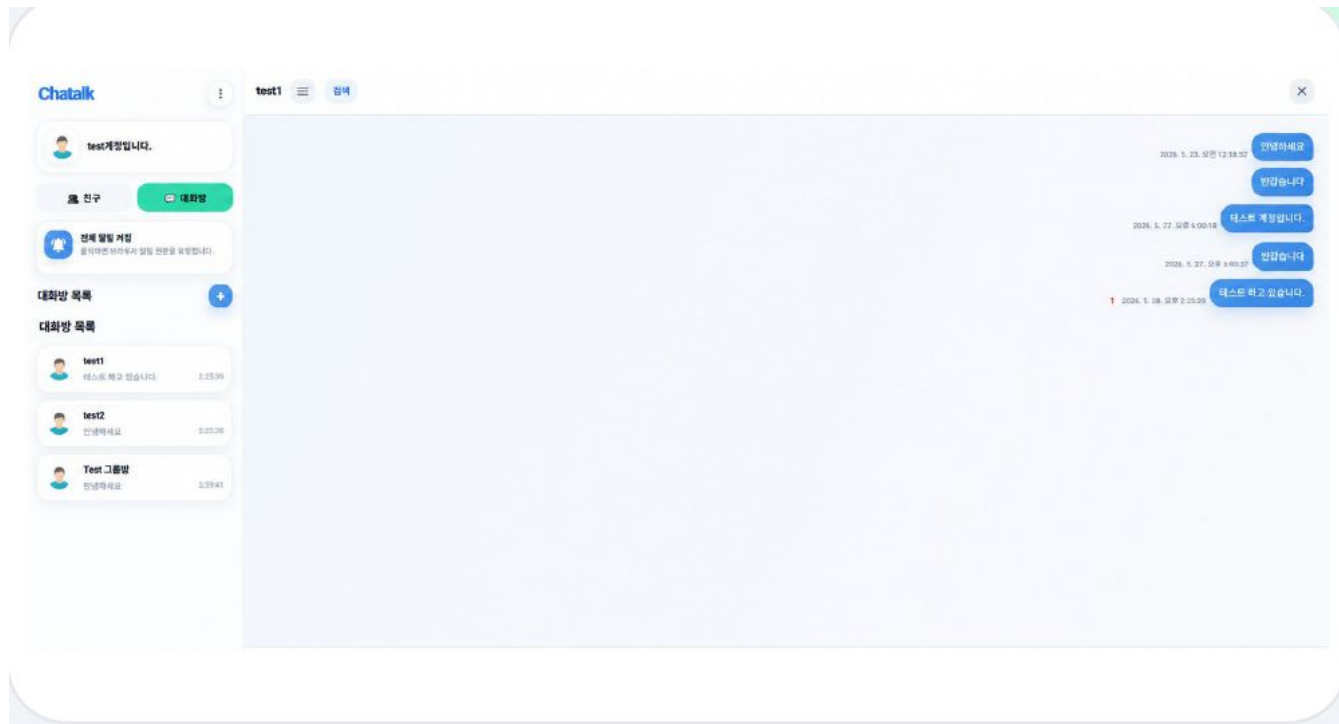
서버 역할 분리

- **API Server:** 채팅 도메인, REST API, WebSocket/STOMP, Redis 연동
- **Auth Server:** 로그인, OAuth2, JWT/쿠키 인증
- **Vue SPA:** 채팅 UI 렌더링 및 WebSocket 연결

핵심 의도: 인증, 실시간 메시징, 화면 렌더링 책임을 분리하고, Nginx에서 경로 기반으로 요청을 라우팅했습니다.

핵심 기능과 서비스 흐름

사용자가 실제로 서비스를 사용하는 흐름 기준으로 기능을 묶었습니다.



서비스 흐름

로그인/OAuth2 → 친구 검색 → 채팅방 진입 →
메시지 전송 → 검색 이동

핵심 기능

실시간 메시징 → WebSocket/STOMP 기반 송수신
읽음

정합성 → lastReadMessageId 기준 상태 관리 연결

유실 복구 → afterMessageId 기반 catch-up 파일

정합성 → 선업로드 + orphan cleanup

검색 이동 → Full-Text + LIKE fallback + context 조회

설계 방향: 기능 목록을 나열하는 데 그치지 않고, 실제 사용 흐름에서 발생하는 정합성·복구·검색 문제를 중심으로 구현했습니다.

읽음/안읽음 정합성 설계

unread라는 같은 이름 안에 서로 다른 기준이 섞이지 않도록 분리했습니다.

문제

채팅방 목록의 안읽은 수는 “내가 읽지 않은 메시지 수”이고, 메시지별 안읽은 수는 “해당 메시지를 아직 읽지 않은 멤버 수”입니다. 둘을 같은 값처럼 처리하면 그룹 채팅에서 정합성이 깨질 수 있습니다.

해결

사용자별 `lastReadMessageId`로 마지막 읽은 위치를 관리하고, 채팅방 목록의 안읽은 수와 메시지별 안읽은 수를 서로 다른 기준으로 계산하도록 분리했습니다.

사용자별 읽은 위치
`lastReadMessageId`

채팅방 목록
현재 사용자 기준 안읽은 수

메시지별 표시
아직 읽지 않은 멤버 수

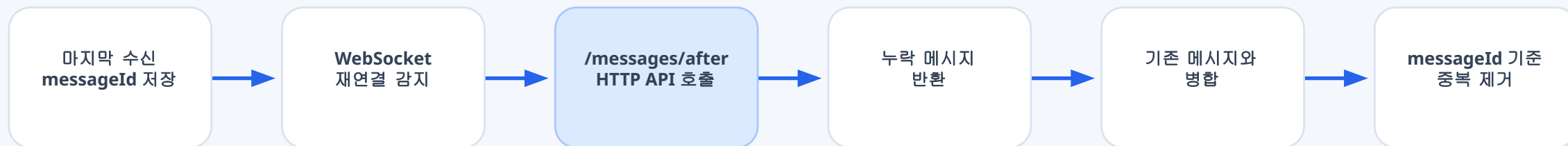
결과: 메시지마다 읽음 row를 저장하지 않고도 사용자별 `lastReadMessageId`를 기준으로 읽은 위치를 관리하고, 채팅방 unread와 메시지별 unread 기준을 분리했습니다.

WebSocket 재연결 후 누락 메시지 복구

실시간 전송과 유실 복구의 책임을 분리했습니다.

문제 상황

WebSocket은 연결이 끊긴 동안 발생한 이벤트를 자동으로 보장하지 않습니다. 브라우저 새로고침, 네트워크 끊김, 서버 재시작 상황에서는 일부 메시지를 실시간 이벤트로 받지 못할 수 있습니다.



WebSocket의 역할

연결된 상태에서 실시간 이벤트를 전달합니다.

HTTP API의 역할

끊긴 동안의 누락 구간을 afterMessageId 기준으로 복구합니다.

Redis Write-Back 기반 메타데이터 동기화

동일 채팅방 메타데이터 동시 갱신에서 발생한 DB 경합을 요청 경로에서 분리했습니다.



① Redis 쓰기 실패 시 DB 직접 반영 fallback

문제

동일 채팅방 DB UPDATE 집중

60건 전송 → 56건 수신 / 46건 DB 저장

Deadlock · Rollback · HikariCP 포화

해결

- Redis Hash 우선 반영
- Dirty Set으로 변경 대상 추적
- Scheduler + Batch Update
- Redis 쓰기 실패 시 DB fallback

비교 결과

sync-db: 수신 56/60 · DB 저장 46/60

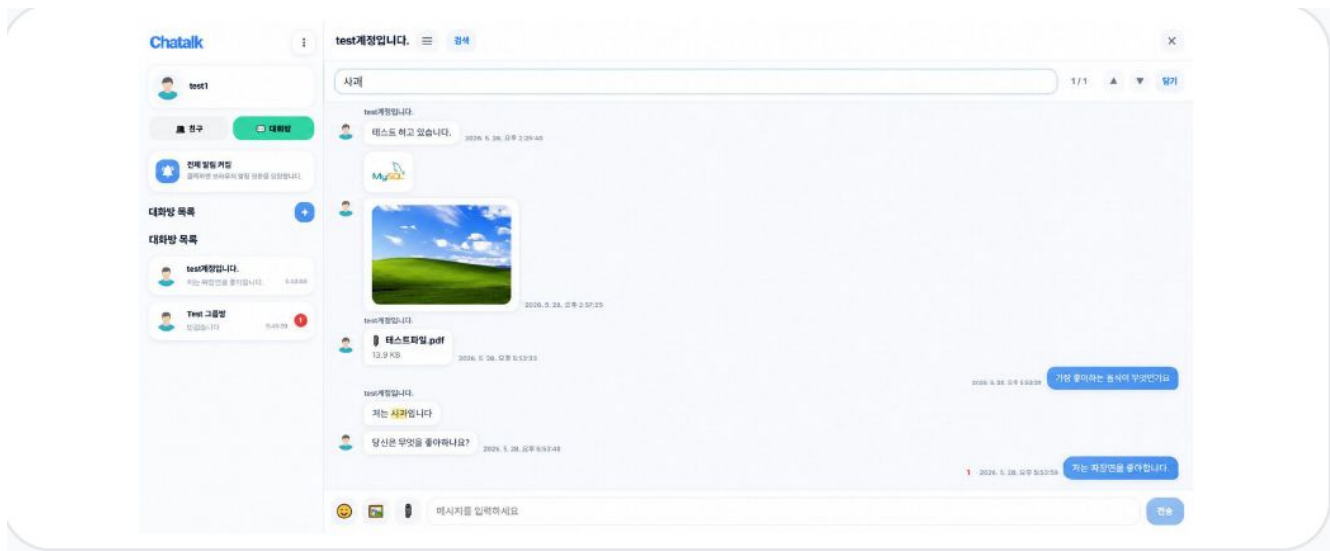
Write-Back: 3회 모두 수신 60/60

DB·Redis 확인 실행: DB 60/60 · Deadlock 증가량 0

· Dirty Set 0

파일 처리와 메시지 검색 UX

실제 채팅 서비스에서 사용자가 체감하는 파일·검색 흐름까지 구현했습니다.



메시지 검색 UX

- 채팅방 단위 키워드 검색을 최신순 cursor 기반으로 반환
- 선택한 messageId 기준 앞뒤 context 조회
- Full-Text Search를 기본으로 사용하되 LIKE fallback 추가
- 검색어 하이라이트와 messages 컨테이너 기준 scrollTop 보정

파일/이미지 메시지 처리

파일을 메시지와 동시에 저장하지 않고 첨부파일로 선업로드한 뒤, 전송 확정 시 FILE/IMAGE 메시지와 attachment를 연결했습니다. 전송 취소로 남는 orphan attachment는 스케줄러가 정리합니다.

핵심 의도

사용자가 체감하는 검색 이동과 파일 전송 흐름을 서비스 수준으로 다듬고, 실패/취소 상황에서 남는 데이터까지 정리하는 구조를 만들었습니다.

검증과 부하 테스트

clientMessageId를 기준으로 publish, DB 저장, WebSocket 수신을 분리해 검증했습니다.



부하 테스트 결과 요약

clientMessageId 기반 정합성 검증 · Publish / DB 저장 / WebSocket 수신 비교

Small

사용자 10명 / 발신자 2명 / 30초 / 1초 간격

Publish	58건
DB 저장	58건 (100%)
예상 수신 / 실제 수신	580건 / 580건
수신 성공률	100%
중복 / 누락	0 / 0

지연 시간 (latency)					
평균	p50	p95	p99	max	
61.48ms	47ms	100ms	494ms	512ms	

Small Long

사용자 10명 / 발신자 2명 / 180초 / 1초 간격

Publish	356건
DB 저장	356건 (100%)
예상 수신 / 실제 수신	3560건 / 3560건
수신 성공률	100%
중복 / 누락	0 / 0

지연 시간 (latency)					
평균	p50	p95	p99	max	
34.63ms	35ms	61ms	73ms	92ms	

Medium

사용자 100명 / 발신자 10명 / 60초 / 500ms 간격

Publish	1,190건
DB 저장	1,190건 (100%)
예상 수신 / 실제 수신	119,000건 / 119,000건
수신 성공률	100%
중복 / 누락	0 / 0

지연 시간 (latency)					
평균	p50	p95	p99	max	
21,450.32ms	21,800ms	39,225ms	41,554ms	42,190ms	

High

사용자 300명 / 발신자 10명 / 60초 / 1초 간격

Publish	590건
DB 저장	590건 (100%)
예상 수신 / 실제 수신	177,000건 / 177,000건
수신 성공률	100%
중복 / 누락	0 / 0

지연 시간 (latency)					
평균	p50	p95	p99	max	
45,329.97ms	45,023ms	86,037ms	89,792ms	91,153ms	



해석 포인트

- 1 Small / Small Long에서는 정합성과 응답성이 모두 안정적
- 2 Medium / High에서도 누락·중복 없이 정합성은 유지됨
- 3 사용자 수 증가에 따라 latency가 크게 증가함
- 4 현재 구조는 정합성은 확보했지만 대규모 팬아웃 환경에서 지연 최적화가 다음 과제임

검증 기준

- 클라이언트 publish 시도 수
- DB chat_message 저장 수
- WebSocket 구독자의 실제 수신 수
- 중복 수신 수 / 누락 수신 수

확인한 점

Small / Small Long / Medium / High 시나리오에서 clientMessageId 기준 DB 저장 성공률과 WebSocket 수신 성공률 100%를 확인했습니다. 중복과 누락은 발생하지 않았습니다.

추가 검증:

- JUnit 테스트 326개 · 실패 0개
- Line Coverage 64.36% · Branch Coverage 65.23%
- JaCoCo 품질 기준 통과

한계

fan-out 규모가 커질수록 latency가 증가했으며, 로컬 부하 테스트라는 한계가 있습니다. 향후 다중 인스턴스 환경 검증과 관측성 보강이 필요합니다.

성과와 한계 및 제출 링크

상세 내용은 Notion과 GitHub에서 확인할 수 있습니다.

핵심 요약

실시간 채팅에서 발생하는 메시지 유실, 읽음 정합성, DB update 경합, 파일 orphan, 검색 UX 문제를 직접 해결했습니다.

성과

- 실시간 메시지 송수신 구조 구현
- 읽음/안읽음 기준 분리와 정합성 설계
- 재연결 후 누락 메시지 catch-up 구조 구현
- 동일 조건 검증에서 Redis Write-Back 적용 후 DB 저장 46/60 → 60/60, Deadlock 증가량 0 확인
- 파일 orphan cleanup 및 메시지 검색 UX 구현
- Redis Pub/Sub 기반 실시간 이벤트 브로드캐스트 적용

한계와 개선 방향

- 단일 서버, 단일 DB/Redis 기준 테스트
- 대규모 fan-out에서 latency 증가
- idempotency key 기반 재시도 안정화 필요
- Redis Pub/Sub 다중 인스턴스 환경 검증 필요
- Prometheus/Grafana 기반 관측성 보강 필요

Service

chataalk.store

API Server

github.com/mino11/live_chat

Front Repository

github.com/mino11/live_chat_front

Auth Server

github.com/mino11/live_chat_auth

Portfolio

[Notion 상세 포트폴리오](#)